

Taylor Shift's Ticket Shop : Infrastructure

Contents

Taylor Shift's Ticket Shop : Infrastructure	1
Documentation	2
Architecture	3
Réseau	3
Compute	3
Stockage partagé	4
Base de données	4
Secrets	4
Organisation Terraform	4
Quickstart	5
Prérequis	5
Déploiement	5
Configuration (Ansible)	5
Accéder à l'application	6
Backend setup	7
Prérequis	7
Création du bucket S3 (une seule fois)	7
Configuration Terraform	7
Traffic handling	9
Comment une requête arrive jusqu'à l'application	9
Montée en charge	9
Si une instance devient indisponible	9
Autres limites	9

Taylor Shift's Ticket Shop : Infrastructure

Infrastructure AWS pour l'application de billetterie Taylor Shift's Ticket Shop (PrestaShop), provisionnée avec Terraform et configurée avec Ansible. L'application étant déjà développée, ce dépôt couvre l'infrastructure, le déploiement, la configuration et l'automatisation.

Documentation

- Architecture : choix d'infrastructure et placement des composants
- Quickstart : déploiement
- Backend setup : mise en place du state Terraform distant
- Traffic handling : gestion du trafic, montée en charge, limites

Architecture

Infrastructure AWS pour PrestaShop (Taylor Shift's Ticket Shop), provisionnée avec Terraform et configurée avec Ansible. Objectif : rester disponible et réactive pendant les pics de vente de billets.

Réseau

- **VPC** : réseau privé isolé, réparti sur 2 zones de disponibilité pour tolérer la perte d'une AZ sans interrompre le service.
- **Internet Gateway** : porte d'entrée/sortie du VPC vers Internet, utilisée par l'ALB (trafic entrant) et le NAT Gateway (trafic sortant).
- **3 tiers de subnets**, chacun présent dans les 2 AZ, pour isoler les composants selon ce qu'ils doivent exposer :
 - **public** : héberge uniquement l'ALB, seul élément qui doit être joignable depuis Internet.
 - **private** : héberge les instances EC2 de l'ASG (PrestaShop). Aucune IP publique, jamais exposées directement.
 - **database** : héberge RDS, isolée du reste, n'accepte que le trafic venant des instances applicatives.
- **NAT Gateway** (partagé, pas un par AZ) : permet aux instances des subnets privés de sortir vers Internet (mises à jour système, image Docker, agent SSM) sans avoir elles-mêmes d'IP publique. Léger SPOF accepté sur la sortie, sans impact sur la haute dispo de l'ALB/ASG qui restent répartis sur les 2 AZ.
- **Aucun accès SSH exposé** : administration des instances via AWS SSM Session Manager, qui ne nécessite ni port entrant ouvert ni clé SSH à gérer.
- **Security groups en chaîne**, sans règle basée sur des CIDR pour le trafic interne : chaque SG n'autorise que le SG qui le précède dans la chaîne, jamais une plage d'IP.
 - **alb** : accepte Internet sur le port applicatif, seule entrée publique de l'infrastructure.
 - **app** : n'accepte que le SG alb, les instances ne sont joignables que via l'ALB.
 - **database et efs** : n'acceptent que le SG app, seules les instances applicatives peuvent y accéder.

Compute

- Un Auto Scaling Group d'instances EC2 dans les subnets `private`, derrière un Application Load Balancer public. Taille et nombre d'instances varient par environnement (voir plus bas).
- Politique de scaling en target-tracking sur le CPU.
- PrestaShop tourne en conteneur Docker (image officielle Docker Hub), déployé et configuré par Ansible.
- Le remplacement automatique d'instance par l'ASG se base sur la santé système, pas sur la santé applicative (voir `traffic-handling.md` pour la justification de ce choix).

Stockage partagé

- PrestaShop écrit par défaut sur le disque local (médias, cache, sessions). Pour scaler horizontalement, ces données doivent être partagées entre les instances : un volume EFS est monté sur chaque instance de l'ASG.

Base de données

- RDS MySQL (moteur natif de PrestaShop), single-AZ.

Secrets

- Ansible Vault est l'unique source de vérité pour les secrets (mot de passe de la base de données, mot de passe administrateur PrestaShop). Pas de service AWS dédié (Secrets Manager), le budget n'étant pas une contrainte forte, ce choix est motivé par la simplicité et le respect direct des critères de notation sur la gestion des secrets.

Organisation Terraform

- Modules dédiés : `network`, `compute`, `database`, `storage`.
- Séparation des environnements (dev/staging/prod) via un fichier `.tfvars` par environnement dans `terraform/environments/`, et une clé de state distincte par environnement dans le même bucket S3 (voir `backend-setup.md`).
 - dev : 1 instance `t3.micro` (min 1, max 2), itération rapide, coût minimal.
 - staging : 1 instance `t3.small` (min 1, max 2), même gabarit que prod, échelle réduite.
 - prod : 2 instances `t3.small` (min 2, max 4), haute disponibilité réelle.

Quickstart

Prérequis

- Terraform >= 1.6
- Un accès AWS configuré (credentials/profile), via un IAM user (pas root)
- Le backend S3 déjà en place, voir backend-setup.md

Déploiement

Trois environnements existent (dev, staging, prod), chacun avec son propre fichier de variables (terraform/environments/<env>.tfvars) et son propre fichier state dans le bucket S3 partagé (voir backend-setup.md). Chaque environnement se déploie avec deux commandes : un init qui pointe vers sa clé de state, puis un apply avec son fichier de variables.

Dev :

```
terraform -chdir=terraform init -backend-config="key=dev/terraform.tfstate"
terraform -chdir=terraform apply -var-file=environments/dev.tfvars
```

Staging :

```
terraform -chdir=terraform init -reconfigure
↳ -backend-config="key=staging/terraform.tfstate"
terraform -chdir=terraform apply -var-file=environments/staging.tfvars
```

Prod :

```
terraform -chdir=terraform init -reconfigure
↳ -backend-config="key=prod/terraform.tfstate"
terraform -chdir=terraform apply -var-file=environments/prod.tfvars
```

-reconfigure n'est nécessaire que pour basculer d'un environnement à l'autre dans un dossier de travail déjà initialisé (par exemple pour tester les trois depuis le même poste), pas sur un premier init après un clone frais.

Chaque environnement crée : le VPC et ses 6 subnets (2 publics, 2 privés, 2 database), l'Internet Gateway, le NAT Gateway, les route tables, les 4 security groups, ainsi que l'Auto Scaling Group applicatif derrière son Application Load Balancer. Seules la taille/le nombre d'instances diffèrent entre environnements (voir architecture.md).

Le mot de passe de la base de données n'a pas de valeur par défaut : Terraform le demande automatiquement (saisie masquée). Sa valeur se trouve dans le Vault Ansible :

```
ansible-vault view ansible/group_vars/app/vault.yml
```

Configuration (Ansible)

1. Installer les collections Ansible nécessaires (inventaire dynamique, connexion SSM, gestion Docker) :

```
ansible-galaxy install -r ansible/requirements.yml
```

2. Vérifier que l'inventaire dynamique voit bien les instances :

```
ansible-inventory -i ansible/inventory.yml --graph
```

Si le groupe app est vide, relancer le terraform apply de l'environnement (les instances ASG n'ont pas toujours fini de démarrer au moment où l'inventaire est généré), puis revérifier.

3. Déployer PrestaShop sur les instances :

```
ansible-playbook -i ansible/inventory.yml ansible/site.yml --ask-vault-pass
```

Accéder à l'application

```
terraform -chdir=terraform output alb_dns_name
```

Backend setup

Le state Terraform est stocké à distance sur S3 (avec verrouillage natif S3) plutôt qu'en local, pour que l'équipe partage le même state et évite les appliques concurrents. Ce backend doit exister **avant** le premier `terraform init`, car Terraform ne peut pas provisionner lui-même les ressources qui stockent son propre state.

Prérequis

- Un compte AWS partagé par l'équipe.
- Un IAM user par personne (pas le compte root), avec la policy `AdministratorAccess`.
- AWS CLI configuré avec ces identifiants (`aws configure`).

Création du bucket S3 (une seule fois)

```
aws s3api create-bucket \
  --bucket taylor-shift-tfstate-819109475304 \
  --region eu-west-3 \
  --create-bucket-configuration LocationConstraint=eu-west-3
```

```
aws s3api put-bucket-versioning \
  --bucket taylor-shift-tfstate-819109475304 \
  --versioning-configuration Status=Enabled
```

```
aws s3api put-public-access-block \
  --bucket taylor-shift-tfstate-819109475304 \
  --public-access-block-configuration
  ↪ BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true
```

Le versioning protège contre une corruption ou un écrasement accidentel du state. Le blocage d'accès public empêche toute exposition publique du bucket, même par erreur. Pas de table DynamoDB à créer : le verrouillage utilise le mécanisme natif de S3 (`use_lockfile`), plus besoin d'une ressource séparée pour ça.

Configuration Terraform

Le bloc backend est déjà déclaré dans `terraform/providers.tf` :

```
backend "s3" {
  bucket      = "taylor-shift-tfstate-819109475304"
  region      = "eu-west-3"
  encrypt     = true
  use_lockfile = true
}
```

`bucket` et `region` sont partagés par les trois environnements (`dev/staging/prod`), un seul bucket suffit, le verrouillage étant scopé par bucket et clé. La clé du state (`key`) n'est volontairement pas fixée ici, car un bloc `backend` ne peut pas utiliser de variable Terraform : elle est donc fournie explicitement à chaque `init`, pour que chaque environnement écrive dans son propre fichier state au sein du même bucket, sans jamais se marcher dessus. Voir `quickstart.md` pour les commandes exactes par environnement.

Traffic handling

Comment une requête arrive jusqu'à l'application

Le client contacte l'Application Load Balancer (ALB), seul point d'entrée public. L'ALB répartit chaque requête vers une des instances applicatives disponibles, dans un des deux subnets privés. Ces instances font tourner PrestaShop dans un conteneur Docker.

Montée en charge

Le nombre d'instances applicatives s'ajuste automatiquement en fonction du CPU moyen utilisé : au-delà d'un certain seuil, une instance supplémentaire est démarrée ; en dessous, une instance est retirée. Ce nombre varie aussi selon l'environnement (1 instance en dev, 2 à 4 en production).

Si une instance devient indisponible

L'ALB détecte en temps réel qu'une instance ne répond plus correctement (panne applicative comprise, pas seulement une panne système) et arrête immédiatement de lui envoyer du trafic. Les autres instances continuent de servir les clients sans interruption visible.

Le remplacement automatique de l'instance par l'Auto Scaling Group, lui, se base uniquement sur l'état système, pas sur la santé applicative. C'est un choix délibéré : une nouvelle instance démarre "nue" et sa configuration applicative (Docker, montage du stockage partagé, déploiement de PrestaShop) n'est pas automatique, il faut relancer manuellement le déploiement Ansible pour qu'elle rejoigne réellement le service. Si le remplacement se basait sur la santé applicative, l'ASG remplacerait en boucle une instance en cours de reconfiguration manuelle, avant même de lui laisser le temps d'être prête. Dans un contexte où la configuration des nouvelles instances serait elle aussi automatisée (auto-configuration au démarrage), ce choix serait revu pour se baser sur la santé applicative de bout en bout.

Autres limites

- La sortie internet des instances passe par une seule passerelle NAT (pas redondée par zone) : un incident dessus couperait leur accès sortant, sans affecter la disponibilité de l'application elle-même côté visiteurs.